MINI PROJECT REPORT

RaahSathi

Railway Reservation System & Passenger Recovery Simulator

Data Structures and Algorithms Project using C++

Submitted by	Radhika Rajput
Roll Number	0126AL251163
Branch / Section	B.Tech AIML / Second Year
Subject	Data Structures and Algorithms
Submitted to	_____

Oriental College of Technology, Bhopal

Academic Session: 2026–2027

Certificate

This is to certify that the mini project titled “RaahSathi: Railway Reservation System & Passenger Recovery Simulator” has been prepared and submitted by the student named on the cover page as part of the Data Structures and Algorithms coursework.

The project demonstrates arrays, pointers, dynamic memory allocation using malloc/free, and a singly linked list in a menu-driven C++ application.

Faculty Signature: _____

Date: _____ Place: Bhopal

Acknowledgement

I would like to express my sincere gratitude to my subject faculty for guidance and support during this mini project. I am thankful to my college for providing an opportunity to apply Data Structures and Algorithms concepts to a practical problem. This project helped me understand how arrays, pointers, linked lists, and dynamic memory management can be used together.

Abstract

RaahSathi is a console-based C++ project that simulates a simplified railway reservation and passenger recovery process. It provides 30 seat slots and supports ticket booking, ticket cancellation, seat-status display, passenger listing, passenger search, disruption simulation, and a recovery report. When a booked seat is blocked, the program attempts to move the affected passenger to another available seat; if none is available, the passenger is marked as waiting. Passenger records are stored in a singly linked list, while seat status is tracked using an array. The project is educational and is not connected to any real railway service.

1. Introduction

Reservation systems need to manage seat availability and passenger information. A disruption, such as a seat becoming unavailable, may require the passenger to be moved or placed on a waiting list. RaahSathi models a simplified version of this workflow so that core data structure concepts can be demonstrated through a console menu.

2. Problem Statement

The problem is to create a simple program that stores passenger information, allocates available seats, supports searching and cancellation, and demonstrates how a passenger can be reassigned when a seat becomes unavailable. The system should also show a summary of available, booked, blocked, and waiting states.

3. Proposed Solution

The solution combines an integer array for seat states with a singly linked list for passenger records. When a passenger books a ticket, the program finds the first available seat and stores the passenger details in a dynamically allocated node. Cancellation removes the passenger node and frees its memory. During a disruption, the selected seat is marked blocked and the affected passenger is moved to another free seat if possible. A recovery report counts the current seat states and passengers on the waiting list.

4. Objectives

- Build a menu-driven C++ railway simulation.
- Allocate seats and store passenger details.
- Allow ticket cancellation and passenger search.
- Display seat status and passenger records.
- Simulate a blocked seat and attempt passenger recovery.
- Use arrays, pointers, malloc/free, and a singly linked list.

5. Tools and Technologies

Item	Use in project

Language	C++
Compiler	Online C++ compiler used for execution/testing
Interface	Console / text menu
Seat storage	Array of 30 seat states
Passenger storage	Singly linked list
Memory management	malloc() and free()

6. Data Structures and Concepts

6.1 Array

The seatStatus array stores the state of each seat. State 0 means available, state 1 means booked, and state 2 means blocked. An array is suitable because each seat has a fixed index from 1 to 30.

6.2 Structure

The Passenger structure groups a passenger's name, age, priority, seat number, and next-node pointer into one record.

6.3 Pointers and Singly Linked List

The head pointer refers to the first passenger node. Each node points to the next node. This allows passenger records to be added and removed while the program is running.

6.4 Dynamic Memory Allocation

malloc() allocates memory for a passenger node during booking. free() releases the node when a booking is cancelled or when the program exits. The code checks whether allocation succeeded before using the node.

7. Modules / Features

Book Ticket

Takes a passenger name, age, and priority. It allocates the first available seat or puts the passenger on the waiting list if all seats are unavailable.

Cancel Ticket

Finds a passenger by name, frees their seat if assigned, unlinks the node, and releases its memory.

View Seat Map

Shows all 30 seats as A (available), B (booked), or X (blocked).

View Passenger List

Displays each stored passenger's name, age, priority, and seat/waiting status.

Search Passenger

Searches the linked list by passenger name and displays the matching details.

Simulate Railway Disruption

Blocks a selected seat. If it was booked, the affected passenger is assigned another available seat or placed on the waiting list.

Recovery Report

Counts available, booked, and blocked seats and the number of passengers waiting.

Exit and Cleanup

Stops the menu loop and frees all remaining passenger nodes.

8. Algorithm / Program Flow

1. Initialize all 30 seats as available and set the linked-list head to NULL.
2. Display the menu and read the user's choice.
3. Run the selected function: booking, cancellation, display, search, disruption, or report.
4. For booking, create a passenger node using malloc() and connect it to the linked list.
5. For disruption, mark the selected seat blocked and attempt to allocate a new seat to the affected passenger.
6. Repeat until the user chooses Exit.
7. Free all remaining nodes before terminating.

9. Testing and Results

The following screenshots show the program outputs captured during testing in an online C++ compiler. Each screenshot is placed under the corresponding test case.

Test 1 — Ticket Booking

Booking confirmation and assigned seat.

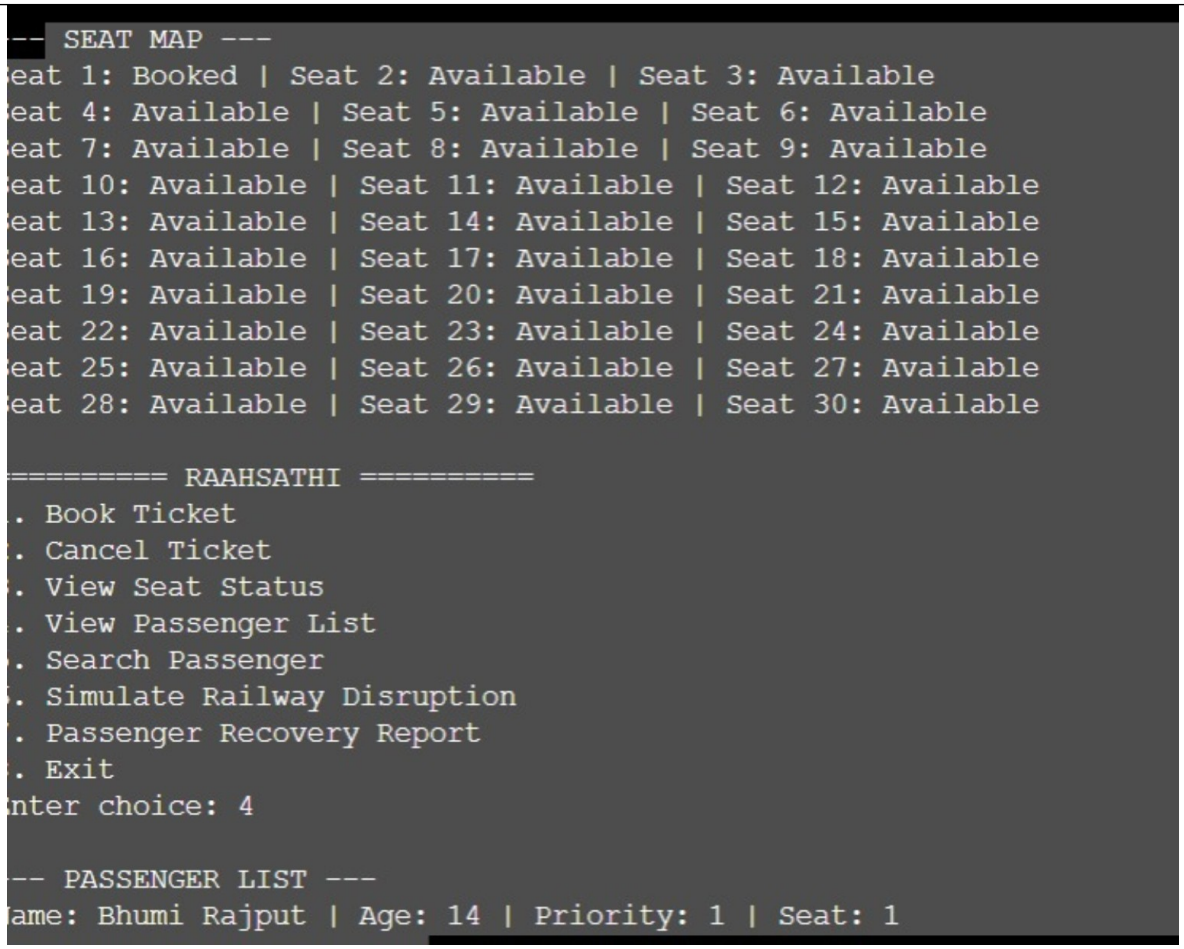
```
========== RAAHSATHI ==========
1. Book Ticket
2. Cancel Ticket
3. View Seat Status
4. View Passenger List
5. Search Passenger
6. Simulate Railway Disruption
7. Passenger Recovery Report
8. Exit
Enter choice: 1
Passenger name: Bhumi Rajput
Age: 14
Priority (1=Urgent, 2=Normal): 1
Ticket confirmed! Seat: 1

========== RAAHSATHI ==========
1. Book Ticket
2. Cancel Ticket
3. View Seat Status
4. View Passenger List
5. Search Passenger
6. Simulate Railway Disruption
7. Passenger Recovery Report
8. Exit
Enter choice: 3
```

Actual output: ticket booking and confirmation.

Test 2 — Seat Map

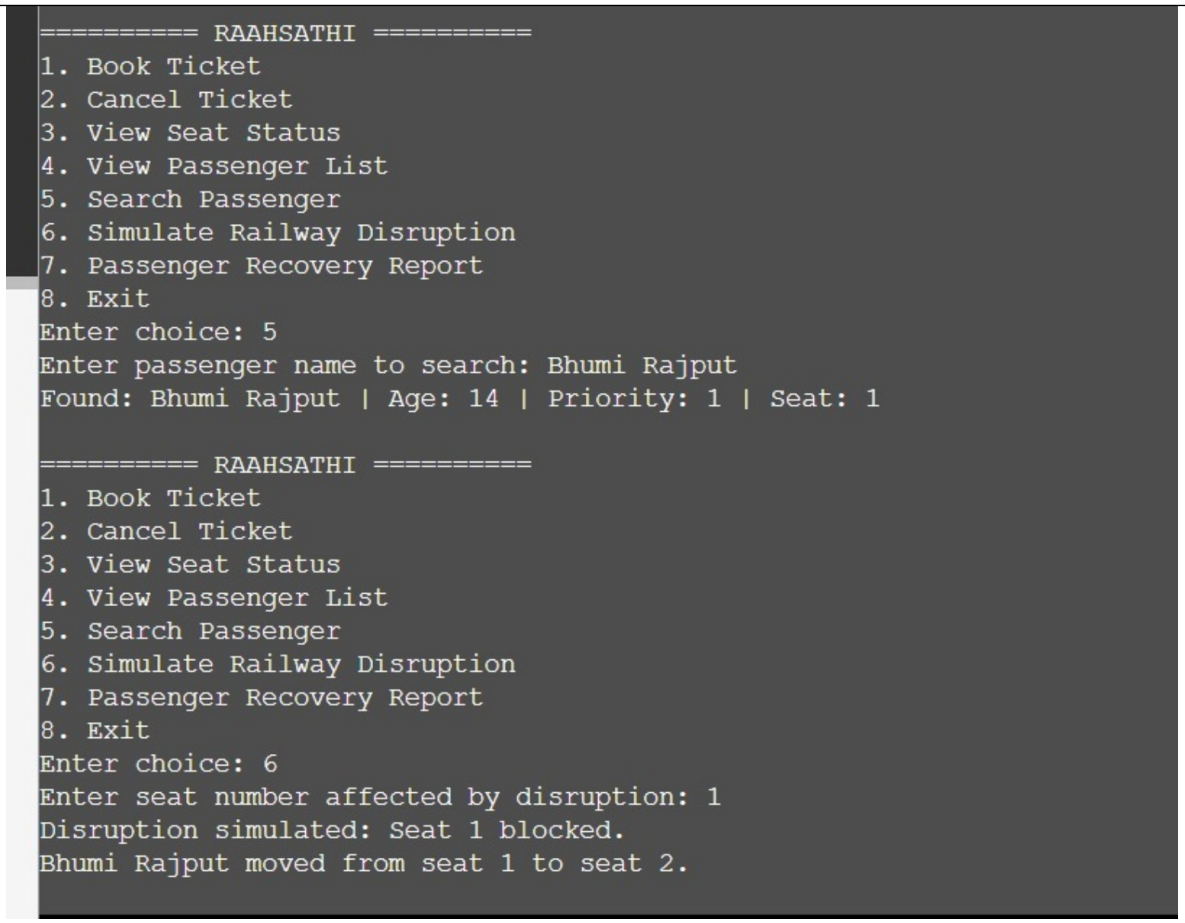
Seat status showing available, booked, and/or blocked seats.



Actual output: seat map and seat availability.

Test 3 — Passenger List and Search

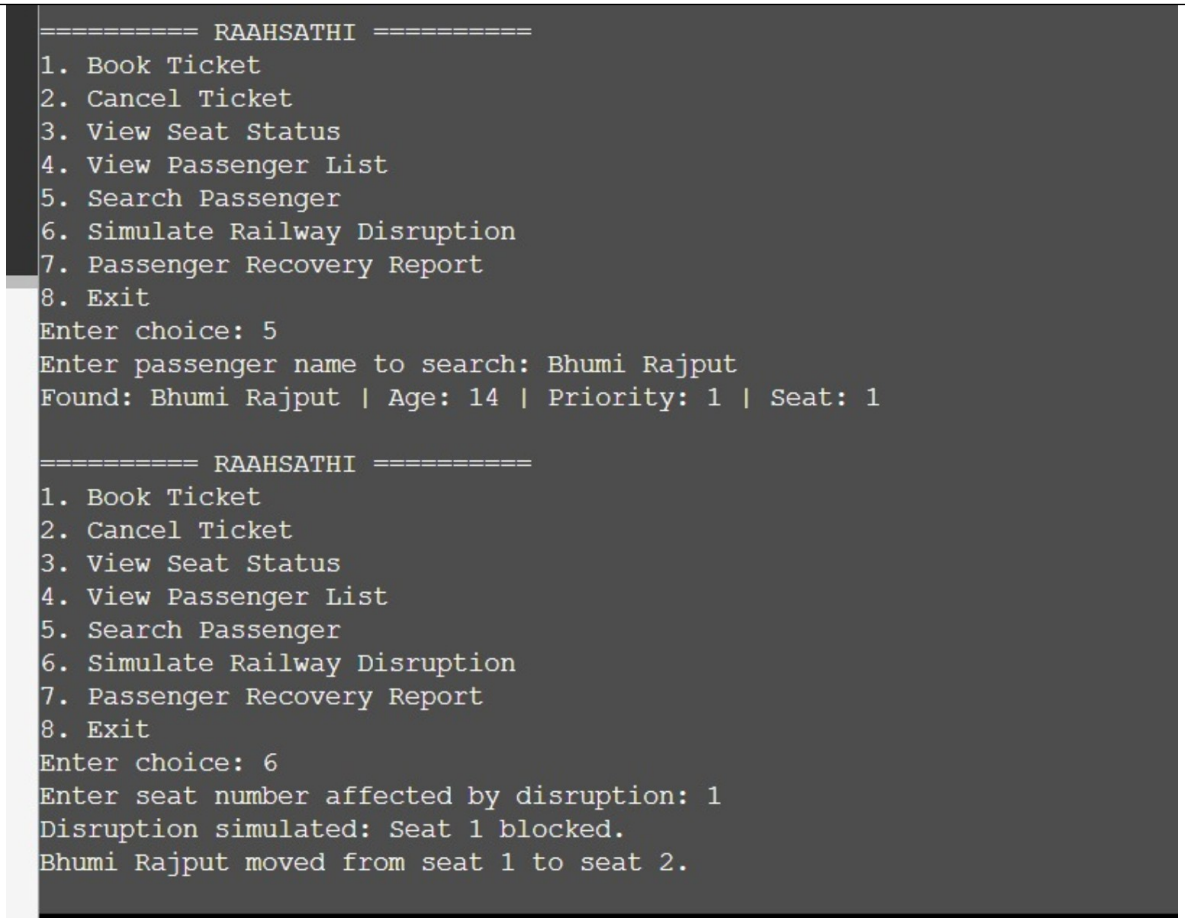
Passenger records and a successful search result.



Actual output: passenger search and disruption simulation.

Test 4 — Disruption Simulation

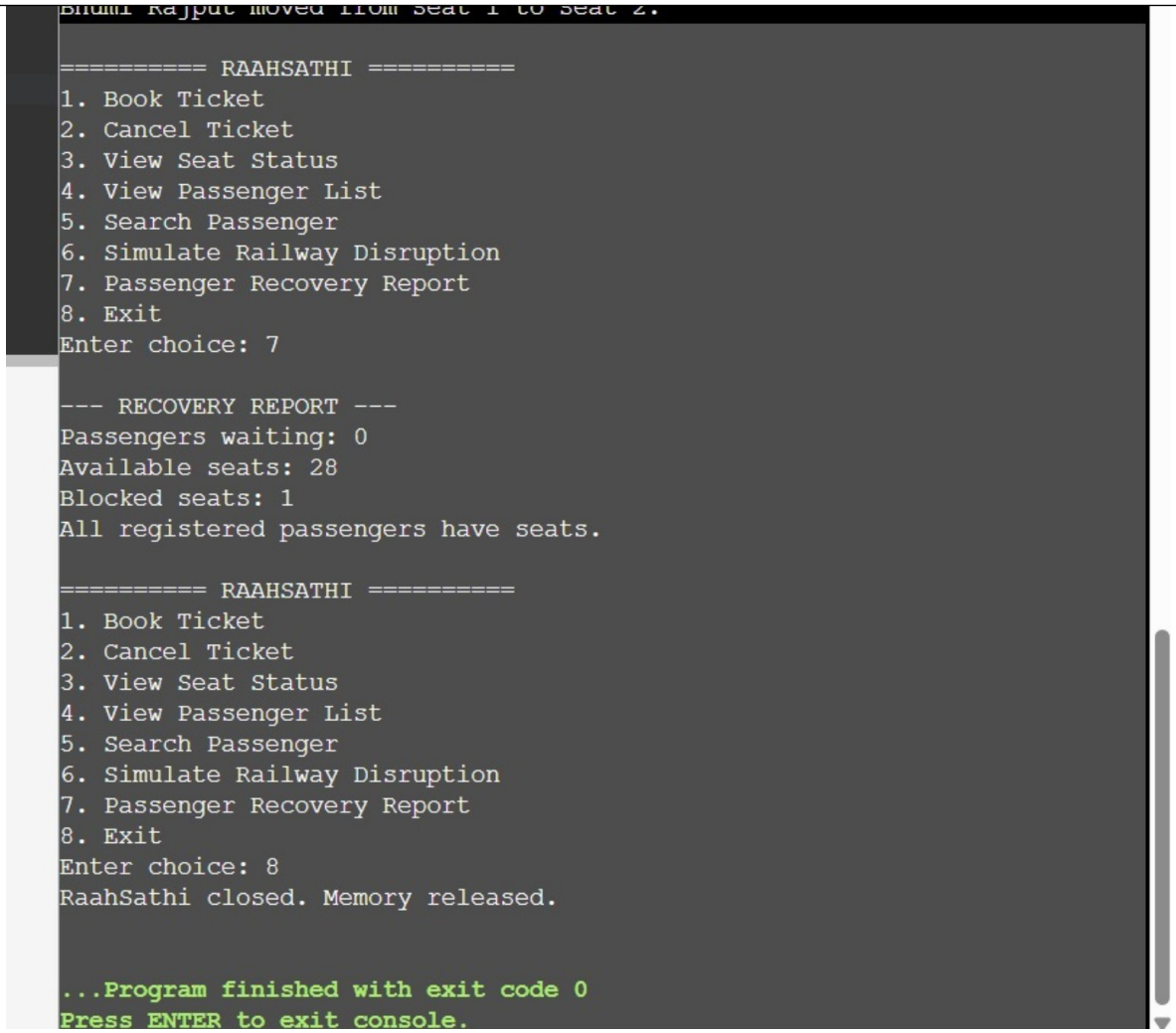
Blocked seat and passenger reassignment or waiting-list result.



Actual output: passenger reassignment after a seat disruption.

Test 5 — Recovery Report and Exit

Summary counts and successful program exit.



Actual output: recovery report and successful exit.

10. Advantages

- Menu-driven and easy to demonstrate.
- Combines multiple DSA concepts in one application.
- Shows dynamic allocation and freeing of passenger nodes.
- Demonstrates a simple recovery action when a seat is blocked.

11. Limitations

- This is a classroom simulation, not a real railway booking system.
- Passenger records are stored in memory and are lost when the program exits.
- Names are entered as one word in this simple version.
- There is no real-time train schedule, payment, login, or official railway connection.

12. Future Scope

- Support full names with spaces and stronger input validation.
- Add file handling or a database for permanent records.

- Add journey date, train number, booking ID, and ticket/PNR generation.
- Improve priority-based waiting-list recovery and add automated tests.
- Develop a graphical user interface.

13. Conclusion

RaahSathi provides a practical demonstration of arrays, pointers, dynamic memory allocation, structures, and singly linked lists in C++. It handles basic passenger and seat operations and demonstrates a simplified response to a seat disruption. The project can be expanded with persistent storage and more realistic railway rules.

References

- Data Structures and Algorithms class notes.
- C++ learning material on arrays, structures, pointers, linked lists, malloc(), and free().
- Testing performed by running the project in an online C++ compiler.

Appendix A — Complete C++ Source Code

The following source code is provided as a separate file named RaahSathi.cpp as well. Use the source file as the executable submission copy. The report copy is included for reference.

```
#include <iostream>
#include <cstdlib>
#include <cstring>
#include <iomanip>
using namespace std;

const int TOTAL_SEATS = 30;
const int NAME_SIZE = 60;

// Seat status: 0 = available, 1 = booked, 2 = blocked
int seatStatus[TOTAL_SEATS + 1];

struct Passenger {
   char name[NAME_SIZE];
   int age;
   int priority;
   int seatNo;          // 0 means waiting for a seat
   Passenger* next;
};

Passenger* head = NULL;

Passenger* findByName(const char name[]) {
   Passenger* current = head;
   while (current != NULL) {
      if (strcmp(current->name, name) == 0)
         return current;
      current = current->next;
   }
   return NULL;
}

Passenger* findBySeat(int seatNo) {
   Passenger* current = head;
   while (current != NULL) {
      if (current->seatNo == seatNo)
         return current;
      current = current->next;
   }
   return NULL;
}

int firstAvailableSeat() {
   for (int i = 1; i <= TOTAL_SEATS; i++) {
      if (seatStatus[i] == 0)
         return i;
   }
   return 0;
}

void addPassenger() {
   char name[NAME_SIZE];
   int age, priority;

   cout << "Enter passenger name (one word): ";
   cin >> setw(NAME_SIZE) >> name;

   if (findByName(name) != NULL) {
      cout << "A passenger with this name already exists.\n";
      return;
   }

   cout << "Enter age: ";
   cin >> age;
```

```
    cout << "Enter priority (1 = high, 2 = normal): ";
    cin >> priority;

    Passenger* node = (Passenger*)malloc(sizeof(Passenger));
    if (node == NULL) {
        cout << "Memory allocation failed.\n";
        return;
    }

    strcpy(node->name, name);
    node->age = age;
    node->priority = (priority == 1) ? 1 : 2;
    node->seatNo = firstAvailableSeat();
    node->next = NULL;

    if (node->seatNo != 0) {
        seatStatus[node->seatNo] = 1;
        cout << "Ticket confirmed for " << node->name
             << ". Seat number: " << node->seatNo << "\n";
    } else {
        cout << "No seat available. Passenger added to waiting list.\n";
    }

    if (head == NULL) {
        head = node;
    } else {
        Passenger* current = head;
        while (current->next != NULL)
            current = current->next;
        current->next = node;
    }
}

void cancelTicket() {
    char name[NAME_SIZE];
    cout << "Enter passenger name to cancel (one word): ";
    cin >> setw(NAME_SIZE) >> name;

    Passenger* current = head;
    Passenger* previous = NULL;

    while (current != NULL && strcmp(current->name, name) != 0) {
        previous = current;
        current = current->next;
    }

    if (current == NULL) {
        cout << "Passenger not found.\n";
        return;
    }

    if (current->seatNo > 0)
        seatStatus[current->seatNo] = 0;

    if (previous == NULL)
        head = current->next;
    else
        previous->next = current->next;

    free(current);
    cout << "Ticket cancelled and passenger record removed.\n";
}

void showSeatMap() {
    cout << "\nSeat status (A=Available, B=Booked, X=Blocked)\n";
    for (int i = 1; i <= TOTAL_SEATS; i++) {
        char symbol = 'A';
        if (seatStatus[i] == 1) symbol = 'B';
        if (seatStatus[i] == 2) symbol = 'X';
```

```
            cout << "Seat " << setw(2) << i << ": " << symbol << "   ";
            if (i % 5 == 0) cout << "\n";
        }
    }

    void showPassengers() {
        if (head == NULL) {
            cout << "No passenger records found.\n";
            return;
        }

        cout << "\nPassenger List\n";
        cout << left << setw(18) << "Name" << setw(8) << "Age"
             << setw(12) << "Priority" << "Seat/Status\n";
        cout << "-----------------------------------------------\n";

        Passenger* current = head;
        while (current != NULL) {
            cout << left << setw(18) << current->name
                 << setw(8) << current->age
                 << setw(12) << (current->priority == 1 ? "High" : "Normal");
            if (current->seatNo == 0)
                cout << "Waiting list\n";
            else
                cout << current->seatNo << "\n";
            current = current->next;
        }
    }

    void searchPassenger() {
        char name[NAME_SIZE];
        cout << "Enter passenger name to search (one word): ";
        cin >> setw(NAME_SIZE) >> name;

        Passenger* p = findByName(name);
        if (p == NULL) {
            cout << "Passenger not found.\n";
            return;
        }

        cout << "Passenger found: " << p->name
             << ", Age: " << p->age
             << ", Priority: " << (p->priority == 1 ? "High" : "Normal");
        if (p->seatNo == 0)
            cout << ", Status: Waiting list\n";
        else
            cout << ", Seat: " << p->seatNo << "\n";
    }

    void simulateDisruption() {
        int seat;
        cout << "Enter seat number to block (1-" << TOTAL_SEATS << "): ";
        cin >> seat;

        if (seat < 1 || seat > TOTAL_SEATS) {
            cout << "Invalid seat number.\n";
            return;
        }
        if (seatStatus[seat] == 2) {
            cout << "This seat is already blocked.\n";
            return;
        }

        Passenger* affected = findBySeat(seat);
        seatStatus[seat] = 2;

        if (affected == NULL) {
            cout << "Seat " << seat << " is now blocked.\n";
            return;
        }
```

```
    affected->seatNo = 0;
    int newSeat = firstAvailableSeat();
    if (newSeat != 0) {
        affected->seatNo = newSeat;
        seatStatus[newSeat] = 1;
        cout << "Seat " << seat << " blocked. " << affected->name
             << " moved to seat " << newSeat << ".\n";
    } else {
        cout << "Seat " << seat << " blocked. " << affected->name
             << " is now on the waiting list.\n";
    }
}

void recoveryReport() {
    int available = 0, booked = 0, blocked = 0, waiting = 0;
    for (int i = 1; i <= TOTAL_SEATS; i++) {
        if (seatStatus[i] == 0) available++;
        else if (seatStatus[i] == 1) booked++;
        else blocked++;
    }

    Passenger* current = head;
    while (current != NULL) {
        if (current->seatNo == 0) waiting++;
        current = current->next;
    }

    cout << "\n===== RECOVERY REPORT =====\n";
    cout << "Total seats       : " << TOTAL_SEATS << "\n";
    cout << "Available seats   : " << available << "\n";
    cout << "Booked seats      : " << booked << "\n";
    cout << "Blocked seats     : " << blocked << "\n";
    cout << "Passengers waiting: " << waiting << "\n";
    cout << "============================\n";
}

void cleanup() {
    Passenger* current = head;
    while (current != NULL) {
        Passenger* nextNode = current->next;
        free(current);
        current = nextNode;
    }
    head = NULL;
}

int main() {
    for (int i = 0; i <= TOTAL_SEATS; i++)
        seatStatus[i] = 0;

    int choice;
    do {
        cout << "\n===== RAAHSATHI: RAILWAY RECOVERY SYSTEM =====\n";
        cout << "1. Book Ticket\n";
        cout << "2. Cancel Ticket\n";
        cout << "3. View Seat Map / Seat Status\n";
        cout << "4. View Passenger List\n";
        cout << "5. Search Passenger\n";
        cout << "6. Simulate Railway Disruption\n";
        cout << "7. View Recovery Report\n";
        cout << "8. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: addPassenger(); break;
            case 2: cancelTicket(); break;
            case 3: showSeatMap(); break;
            case 4: showPassengers(); break;
```

```
            case 5: searchPassenger(); break;
            case 6: simulateDisruption(); break;
            case 7: recoveryReport(); break;
            case 8: cout << "Program finished. Thank you!\n"; break;
            default: cout << "Invalid choice. Please try again.\n";
        }
    } while (choice != 8);

    cleanup();
    return 0;
}
```